



Navi 个人导航站

飞牛 fnOS 部署指南

单容器 · 零依赖 · 双栈 · 服务发现 · 图床库 · 密码保护

适用环境：飞牛 fnOS (Debian 内核) + Docker / Docker Compose

适用版本：Navi 1.x (node:22-alpine 镜像, 零 npm 依赖)

文档日期：2026 年 9 月 19 日

目录

1. 部署前必读：三条会踩坑的硬规则

- 2. 环境准备 (Docker / SSH / 存储空间)
- 3. 目录规划
- 4. 把源码放到 NAS 上
- 5. 方式一: SSH 命令行部署 (推荐)
- 6. 方式二: 飞牛 Docker 图形界面部署
- 7. 端口与存储映射说明
- 8. 网络配置 (双栈 / 反代 / 安全)
- 9. 环境变量配置
- 10. 可选: 开启服务发现
- 11. 启动与验证
- 12. 常见问题与排查 (14 项)
- 13. 日常运维与升级回滚
- 14. 部署检查清单
- 15. 命令速查

1. 部署前必读: 三条会踩坑的硬规则

飞牛 fnOS 的 Docker 与通用 Linux 无本质区别, 但下面三点如果忽略, 会分别导致「构建不出来」「保存报错」「站点裸奔」。建议先读完这一节再动手。

硬规则	原因与做法
① 必须按「整目录」挂载数据	写配置采用 原子写 (先写 <code>.tmp</code> 再 <code>rename</code> 覆盖)。若只挂载单个 <code>config.json</code> , <code>rename</code> 会跨越两个挂载点, 在飞牛的存储卷上会稳定触发 <code>EBUSY: resource busy or locked</code> , 表现为 网页上点「保存」永远失败 。  正确写法: <code>- /vol1/1000/docker/navi/data:/app/data</code> (挂目录, 不挂文件)
② 必须提供访问密码	仓库的 <code>docker-compose.yml</code> 用的是 <code>\${NAVI_PASSWORD:?...}</code> 强制校验: 缺少该变量时 compose 会直接报错退出 。这是刻意设计——避免静默退化成「无密码开放访问」(任何拿到地址的人都能进编辑模式改你的配置)。 做法: 在 <code>compose</code> 同目录放一份 <code>.env</code> (见第 6 节)。
③ 没有预构建镜像, 需要自己 build	本项目 没有发布到任何镜像仓库 , <code>image: navi:latest</code> 指的是「本机构建出来的镜像」。所以要么在 NAS 上 <code>docker build</code> (需联网拉取 <code>node:22-alpine</code>), 要么在别的电脑构建后导出再导入 (见第 5 节方式 B)。

好消息: 容器内已内置示例配置, **不挂载也能跑起来**。你可以先用最简命令验证镜像没问题, 再补挂载做正式部署。

2. 环境准备

1 确认 Docker 已安装并已选定存储空间

首次打开飞牛的「Docker」应用时，系统会提示选择 Docker 的存储位置（建议选 **SSD 所在存储空间**，机械盘上构建镜像会明显更慢）。选定后会该存储空间根目录生成一个 `docker` 文件夹，用于存放镜像与运行环境。

路径	用途
<code>/vol1/docker</code>	Docker 自身的镜像 / 运行环境 / 网络 / 插件（由飞牛自动管理）
<code>/vol1/1000</code>	你的用户目录（ <code>1000</code> 是飞牛默认用户 ID），我们放 compose 与数据的位置
<code>/vol2、/vol3...</code>	其它存储空间，可按需替换下面的 <code>/vol1</code>

2 开启 SSH（命令行部署方式需要）

飞牛默认关闭 SSH。进入 **设置** → **远程访问** → **SSH**，勾选启用（建议同时改掉默认 22 端口）。之后用任意终端登录：

```
ssh 你的用户名@飞牛NAS的IP
# 登录后切换到 root（飞牛的 docker 命令需要 root 权限）
sudo -i
```

3 验证 Docker 可用

```
docker --version      # 例：Docker version 27.x.x
docker compose version # 例：Docker Compose version v2.x.x
```

如果 `docker compose` 提示不认识该子命令，说明只有旧版 `docker-compose`（带横线）。把本文所有 `docker compose` 替换为 `docker-compose` 即可。

3. 目录规划

约定一个固定目录，后面所有命令都以它为例（请按自己的存储空间替换 `/vol1`）：

```
/vol1/1000/docker/navi/      ← 项目目录（compose、.env、源码）
├─ docker-compose.yml        仓库自带，直接沿用
├─ .env                      ★ 放真实密码，需自己创建
├─ Dockerfile                仓库自带
├─ server.js                 后端（零依赖）
├─ discovery.js              服务发现模块（容易漏！见下方提醒）
├─ public/                   前端资源
├─ data/                     ← 持久化数据（挂载到容器 /app/data）
│   └─ config.json           你的导航链接配置
│   └─ uploads/              图床库 / 卡片 Logo 图片
```

容易漏的一个文件： `discovery.js` 。它不在 `public/` 里，复制源码时很容易只拷 `server.js + public/` 而把它落下——结果就是容器启动直接报 `Cannot find module './discovery'` 。**上传源码后务必确认它存在。**

哪些文件不必上传： `.git/`、`test/`、`docs/`、`scripts/`、`node_modules/`、`public/config.json`（你的真实配置，会被 `.dockerignore` 排除）、`public/uploads/`。仓库自带的 `.dockerignore` 已经帮你在构建时排除了这些，所以构建上下文很小，构建速度不受影响。

4. 把源码放到 NAS 上

三种方式，任选一种。核心要求只有一条：**第 3 节列出的文件都要在 `/vol1/1000/docker/navi/` 下。**

方式	操作	适用场景
A. Git 克隆 (最省事)	<pre>cd /vol1/1000/docker git clone 你的仓库地址 navi</pre> 仓库地址若为私有，需要先在飞牛上配好凭据（SSH Key 或 PAT）。若还没推到 GitHub，参考 README 的「开源发布到 GitHub」章节。	源码已推到 Git 仓库； 后续升级只要 <code>git pull</code>
B. 文件管理上传	在飞牛「文件管理」中切到 管理员视角 → 进入 <code>/vol1/1000/docker/</code> → 新建文件夹 <code>navi</code> → 把本地项目打包成 zip 上传 → 右键 解压到当前目录 （注意别解压出多层目录）。	不想在 NAS 上折腾 Git，或 NAS 访问 GitHub 慢
C. SMB 共享拷贝	把 <code>/vol1/1000/docker/navi</code> 设为共享文件夹，在电脑上通过「映射网络驱动器」直接把整个项目目录拷进去。	局域网环境，且要经常改文件

不要用「单文件逐个上传」代替整目录拷贝——尤其是 `public/` 目录里的 `js/app.js`、`css/style.css` 等，漏一个就会导致页面样式或交互异常。

5. 方式一：SSH 命令行部署（推荐）

这是最可控的方式：日志实时可见，出错时能立刻定位。全程约 3~5 分钟（取决于网络拉取基础镜像的速度）。

1 建立目录

```
sudo mkdir -p /vol1/1000/docker/navi/data/uploads
sudo chown -R 1000:1000 /vol1/1000/docker/navi # 属主交给你的用户，方便文件管理里操作
cd /vol1/1000/docker/navi
```

2 放入源码

按第 4 节任选一种方式把项目文件放进来，然后确认关键文件齐全：

```
ls -l server.js discovery.js Dockerfile docker-compose.yml public/index.html
# 五项全部列出即可；缺 discovery.js 会在启动时报 Cannot find module
```

3 初始化导航配置

```
cp public/config.example.json data/config.json
# 校验一下是合法 JSON (飞牛自带 node 的可能性低，用 python3 兜底)
python3 -c "import json;json.load(open('data/config.json'));print('config.json OK')"
```

4 创建 .env 写入访问密码

这是**唯一必填**的配置。仓库已提供 `.env.example` 模板：

```
cp .env.example .env
# 编辑 .env，把 CHANGE_ME 换成你的强密码
vi .env          # 或用 nano .env

# 确认写入成功（只显示变量名，不显示值）
grep -q '^NAVI_PASSWORD=.*' .env && echo ".env 已设置密码"
```

不想留明文？用哈希替代。注意比对的是 `sha256("navi-site:" + 密码)`，**必须带 navi-site: 前缀**，直接用裸 `sha256` 会永远登录失败（返回 401 且不报错，极易误判成密码写错）：

```
# 用任意装了 Node 的电脑执行（飞牛上通常没装 node）
node -e "console.log(require('crypto').createHash('sha256').update('navi-site:'+process.argv[1]).digest('hex'))" '你的强密码'
# 把输出的 64 位哈希填进 .env 的 NAVI_PASSWORD_HASH=，并删掉或注释掉 NAVI_PASSWORD= 那一行
```

5 构建镜像

```
sudo docker build -t navi:latest .
# 看到 Successfully tagged navi:latest 即成功（首次需联网拉取 node:22-alpine，约 50MB）
```

构建卡在拉取阶段？说明 NAS 到 Docker Hub 的网络不通。在飞牛 **Docker 应用** → **设置** → **镜像加速**里配置一个国内加速地址后重试；或改用下面的「方式 B：跨机导入镜像」，完全不需要在 NAS 上联网构建。

方式 B：在别的电脑构建后导入（NAS 无法联网时）

在你的电脑（已装 Docker Desktop / Docker）上，进入项目目录执行：

```
# 电脑上：构建并导出为 tar (Windows PowerShell 用双引号包路径)
docker build -t navi:latest .
docker save navi:latest -o navi-image.tar
```

```
# 把 navi-image.tar 上传到 NAS 的 /vol1/1000/docker/navi/ 后:
sudo docker load -i /vol1/1000/docker/navi/navi-image.tar
sudo docker images | grep navi      # 确认 navi:latest 已在列表中
```

导入镜像后，记得把 `docker-compose.yml` 里的 `build: .` 那一行删掉或注释掉，只保留 `image: navi:latest`，否则 `compose` 仍会尝试重新构建。

6 启动容器

```
sudo docker compose up -d

# 查看状态与日志
sudo docker compose ps
sudo docker logs --tail 30 navi
```

日志应出现类似输出（端口、配置路径、鉴权状态）：

```
[navi] listening on [::]:80
[navi] config file: /app/data/config.json
[navi] auth: ENABLED (ttl 72h)
```

7 验证

见第 9 节的完整验证清单。

6. 方式二：飞牛 Docker 图形界面部署

不想碰 SSH 的话，飞牛自带的 Docker 应用支持 Compose 项目，全程鼠标操作。但有一个关键限制必须先处理。

GUI 的 Compose 项目通常不支持 `build:`（构建）——它只会去拉取现成镜像。而本项目的镜像是本地构建出来的，不在任何仓库里。因此 GUI 方式的正确姿势是：**先构建镜像，再用 GUI 启动**：

```
sudo docker build -t navi:latest /vol1/1000/docker/navi
```

（这一条命令仍需 SSH 执行一次；或者用第 5 节的「跨机导入镜像」把 tar 传进来 `docker load`。）

1 准备目录与文件

在「文件管理（管理员视角）」中建好 `/vol1/1000/docker/navi/`，放入源码，并**手动创建** `data/` 目录与其中的 `config.json`（把 `public/config.example.json` 复制过去并改名为 `config.json`）。

2 新建 Compose 项目

打开「Docker」应用 → 左侧 **Compose** → **新增项目**：

字段	填写内容
项目名称	navi
路径	/vol1/1000/docker/navi（选择该目录后，界面一般会提示「使用现有的 docker-compose 配置来创建项目」）
创建后立即启动	☒ 勾选

3 粘贴 GUI 版 compose 配置

仓库自带的 `docker-compose.yml` 依赖 `.env` 提供密码。GUI 下更简单的做法是**把密码直接写进环境变量**，并把 `build: .` 换成已有的本地镜像。可直接粘贴下面这份（只有 4 处需要按你的情况改）：

```
services:
  navi:
    image: navi:latest           # 用第 5 节构建/导入好的本地镜像
    container_name: navi
    restart: unless-stopped
    environment:
      - SITE_TITLE=Nav1
      - SITE_SUBTITLE=个人导航站
      - NAVI_PASSWORD=改成你的强密码
      - SESSION_TTL_HOURS=72
      - NAVI_CONFIG_PATH=/app/data/config.json
      - NAVI_UPLOAD_DIR=/app/data/uploads
    ports:
      - "8080:80"               # 左侧 8080 若被占用，换成其它空闲端口
    volumes:
      - /vol1/1000/docker/navi/data:/app/data  # 整目录挂载，勿改成单文件
```

后续要改这份配置（换端口 / 改密码）时，飞牛 GUI 的生效流程是三步：**停止项目** → **修改 yml** → **清除项目** → **再启动**。只改 yml 不重新创建，配置不会生效。

若你更愿意用仓库原版 `docker-compose.yml`：在 GUI 里把 `build: .` 注释掉，并在该目录下用文件管理新建一个 `.env`（内容一行 `NAVI_PASSWORD=你的密码`）。Docker Compose 会自动读取同目录的 `.env`。

在 GUI 里编辑 yml 最容易踩的坑：YAML 缩进

飞牛的 Compose 编辑器会实时校验语法，缩进一旦不齐，就会在出问题的那几行画红线。三条铁律：

1. 同一个 `environment:` 下，所有 `-` **必须严格对齐在同一列**。本项目原版是 `6 个空格 + "-" + 1 个空格 + 键=值`，取消注释时不要连带空格一起删错——多一格、少一格都会报错。
2. 同一个 `environment:` 里**不能混用**「列表式」`- 键=值` 与「映射式」`键: 值`，两种写法必须整段统一，混用必然报 `bad indentation of a mapping entry`。
3. 值里带冒号（如 `http://10.10.10.18`）时建议**整体加引号**，可以规避各种解析器的歧义判断。

以「内网 / 外网地址基址」这两行为例，要启用就写成下面这样（重点是 `-` 与上面的同级项严格同列）：

```
- "NAVI_LAN_HOST=http://10.10.10.18"
- "NAVI_WAN_HOST=https://dam123.top"
```

更省事的做法是这两行干脆不启用。不设置时服务发现会自动探测 NAS 的本机 IP；只有探测不准（NAS 有多网口 / 虚拟网卡）时才需要手动指定。

拿不准有没有写歪，可以在电脑上（不用在 NAS 上）跑一次自检，它会把最终解析出的环境变量原样打印出来：
`NODE_PATH=<含 js-yaml 的 node_modules> node scripts/check-compose.cjs`；它会明确指出哪一行被「静默折叠」了。

7. 端口与存储映射说明

端口

项	值	说明
容器内监听端口	80	镜像通过 <code>PORT=80</code> 固定，无需修改
宿主映射端口	8080	宿主机访问端口，可自由修改，如 <code>"18080:80"</code>
访问地址	<code>http://飞牛NAS的IP:8080</code>	局域网内直接用飞牛 IP

部署前先确认 8080 没被占用（飞牛自身服务、其它容器都可能占用）：

```
sudo ss -tlnp | grep -E '':8080\b' # 无输出即端口空闲
```

存储映射

宿主机路径	容器内路径	存放内容
<code>/vol1/1000/docker/navi/data</code>	<code>/app/data</code>	导航配置 <code>config.json</code> + 图床库 / Logo 图片 <code>uploads/</code> 。整个目录一起挂载，不要拆成单文件

这个目录丢了 = 数据全丢。网页上的「备份」按钮只导出 `config.json`，不打包图片实体。迁移或重装前，请一并复制 `data/uploads/`。图片缺失时卡片会自动回退为字母图标，不会报错，所以容易察觉不到。

重建容器时要不要清空这个目录？——不要

先分清两样东西，它们不是一个概念：

对象	是什么	重建 / 清除时
镜像 <code>navi:latest</code>	程序代码本身	随便删、随便重建，删掉不心疼（随时能从源码重新 <code>build</code> ）

数据目录 <code>data/</code>	<code>config.json + uploads/</code> ，你全部导航卡片与图片	不可再生，只此一份——重建镜像与容器都不会动它
-------------------------	---	-------------------------

原因是这一行是 **目录挂载 (bind mount)**：`- /vol1/1000/docker/navi/data:/app/data`。Docker 只负责把宿主机目录「接」进容器，**不管理它的生命周期**。所以下面这些操作都不会影响数据：

```
# 以下操作都对 data/ 无影响，放心执行
sudo docker build -t navi:latest /vol1/1000/docker/navi # 重建镜像
sudo docker rm -f navi # 删除容器
sudo docker compose up -d --force-recreate # 重建容器
飞牛 GUI：停止项目 → 清除项目 → 再启动 # 清掉的是容器与项目定义
```

- 唯一会丢数据的三种情况（务必避开）：**
- 1. 在「文件管理」里**手动删除** `data/` 目录（或改名后忘记改回）；
 - 2. `docker compose down -v` 或 `docker volume rm`——加了 `-v` 会连卷一起删。**本项目用的是路径挂载，`down -v` 并不会删它**；但如果你当初把它写成了具名卷（`- navi-data:/app/data`），就会连数据一起删掉。检查 `yaml` 里写的是 `/vol1/...` 这种路径就没问题；
 - 3. 重装系统 / 换盘时没有把 `data/` 一起迁移走。

每次升级前的标准动作（一条命令，比网页导出更全，含图片）：

```
sudo tar czf /vol1/1000/navi-backup-$(date +%F).tar.gz -C /vol1/1000/docker/navi data
# 动手前先确认目录里有东西
ls -l /vol1/1000/docker/navi/data/ # 应看到 config.json 与 uploads/
```

权限

镜像未设置 `USER`，容器内默认以 **root** 运行，因此对挂载目录一定有写权限——这是它能开箱即用的原因。副作用是：**容器写入的文件在宿主机上属主为 `root:root`**，飞牛「文件管理」（以 1000 用户身份）可能无法直接编辑或删除这些文件，提示权限不足。

如果需要让文件管理也能操作，有两种办法：

```
# 办法一（推荐）：直接在 SSH 里改属主
sudo chown -R 1000:1000 /vol1/1000/docker/navi/data

# 办法二：让容器以你的用户身份运行 — 在 compose 的 navi 服务下增加一行
user: "1000:1000"

# 前提：data 目录属主必须是 1000:1000，否则容器内会写不进去（保存失败）
```

8. 网络配置

项	说明
---	----

网络模式	保持默认的 bridge 即可，通过 <code>-p 8080:80</code> 对外暴露。本项目是单端口纯 HTTP 服务， 不需要 host 模式 ，也不需要 macvlan
IPv4 / IPv6 双栈	服务端默认监听 <code>::</code> （同时接受 IPv4 与 IPv6）。宿主机开启 IPv6 时，Docker 的端口映射会同时绑定两种协议栈， 无需额外配置 。验证： <code>curl -g "http://[你的IPv6地址]:8080/api/health"</code>
局域网访问	同网段设备直接访问 <code>http://NAS的IP:8080</code> 。建议在飞牛确认防火墙未拦截该端口
外网访问	请 不要 直接把 8080 端口映射到公网。正确做法是使用反向代理（飞牛可装的 Lucky / Nginx Proxy Manager / Caddy）并配置 HTTPS 证书；服务在 HTTPS 下会自动为会话 Cookie 加 Secure 标记
容器互访	服务发现要列出其它容器时，需接入 Docker Socket 或远程 API —— 见第 10 节

反代时的注意点：把 `/` 整个路径转发到 `http://NAS的IP:8080`，并**保留 Host 头**。服务会用请求的 Host 推断内网地址基址，Host 被改写可能导致服务发现生成的地址不对；这种情况可以显式设置 `NAVI_LAN_HOST=NAS的IP`。

9. 环境变量配置

上面「GUI 版 compose」里已经给了最小可用组合。下表是飞牛部署时**真正会用到的**变量（完整列表见 README）：

变量	作用	建议值
NAVI_PASSWORD	访问密码。 必填 （compose 会强校验）	强密码
NAVI_PASSWORD_HASH	密码的加盐哈希，替代明文（更推荐）	64 位哈希
NAVI_USERNAME	设置后启用「用户名 + 密码」联合登录	admin（可选）
SESSION_TTL_HOURS	登录会话有效期（小时）	72
SITE_TITLE / SITE_SUBTITLE	覆盖站点标题 / 副标题（仅展示层，不写回配置）	按喜好
NAVI_CONFIG_PATH	配置文件路径	/app/data/config.json
NAVI_UPLOAD_DIR	Logo / 图床库目录	/app/data/uploads
NAVI_LAN_HOST	服务发现生成的内网地址基址	NAS的局域网IP
NAVI_WAN_HOST	服务发现生成的外网地址基址	你的域名（可选）
NAVI_ICON_PROBE	设为 <code>0</code> 关闭图标在线探测，服务发现更快	离线环境设 <code>0</code>

NAVI_ICON_PROBE_BASE	图标探测的 CDN 基址，可指向自建镜像（须保持上游目录结构）	用默认值即可
DOCKER_SOCKET	启用容器发现（见第 10 节）	/var/run/docker.sock

不要把端口映射与服务监听的端口搞混。改「访问端口」只改 compose 里 ports 左侧的数字（如 8080 → 18080）；容器内始终是 80，不要去改 PORT 环境变量，否则端口映射会对不上，页面打不开。

密码的三种写法（择一）

方式	compose 环境变量	适用
明文密码	- NAVI_PASSWORD=强密码	最简单，仅本机可见
哈希密码 (推荐)	- NAVI_PASSWORD_HASH=64位哈希	环境变量里也不留明文
用户名 + 密码 (更安全)	- NAVI_USERNAME=admin - NAVI_PASSWORD_HASH=64位哈希	暴露到公网时推荐

启用后的行为：未登录访问任何页面 / 静态资源 / API 一律跳登录页（API 返回 401）；同一 IP 连续登录失败 5 次锁定 1 分钟；会话令牌为 HMAC 签名的 HttpOnly Cookie；页头提供「退出」按钮，登出会服务端吊销该令牌。

「用户名 + 密码」联合登录时，用户名或密码任一错误都只提示「用户名或密码错误」，不区分是哪一项——这是为了防止用户名被逐个试出来。

10. 可选：开启服务发现（自动识别容器）

本项目支持扫描 NAS 上正在运行的容器，自动识别端口、匹配图标并生成导航卡片。不开启也完全不影响其它功能——只影响编辑模式里「服务发现」按钮能否列出容器。

1 挂载 Docker Socket

在 docker-compose.yml 的 volumes 下取消注释这一行：

```
- /var/run/docker.sock:/var/run/docker.sock:ro
```

然后重建容器：

```
sudo docker compose up -d --force-recreate
```

安全提示（务必理解再开）：挂载 Docker Socket 后，容器内进程即拥有等价于宿主机 root 的 Docker 控制权——可以读取所有容器环境变量（其中常含数据库密码、API Key），甚至可以起一个特权容器来读写宿主机任意文件。

另外要注意：`:ro` 只是把 `socket` 文件标记为只读，并不能限制通过该 `socket` 发起的 `Docker API` 调用。它更多是语义标记，不要误以为加了 `:ro` 就安全了。

因此：仅在你信任镜像来源、且该端口只在内网可访问时开启；若要暴露公网，建议不挂 `socket`（或改用下面的远程 `API` 方式并只读开放）。

替代方案：连远程 Docker API（不挂 socket）

如果 NAS 上另有 Docker 主机，或你不想把 `socket` 交给容器，可改用 `TCP` 方式，仅需环境变量：

```
- DOCKER_HOST_NAME=192.168.1.20
- DOCKER_HOST_PORT=2375
```

开放 `Docker TCP API`（2375）本身是未加密且无认证的，同样等于交出 `root`。若必须使用，请只在隔离内网开放，并考虑用 2376 + `TLS`。相比之下，本机挂 `socket :ro` 反而更收敛一些。

只想看容器、不想看 NAS 本机端口？设置 `NAVI_SCAN_LOCAL=0`。容器内没有 `/proc/net/tcp`，本机扫描会走 `netstat` 兜底，结果通常为空白——关掉可以少一点噪声。

离线环境建议同时设置 `NAVI_ICON_PROBE=0`，避免扫描时联网探测图标拖慢速度（未命中的图标会自动回退为字母图标，不影响使用）。

11. 启动与验证

1 容器在运行且未反复重启

```
sudo docker compose ps
# 期望: STATUS 显示 Up X seconds/minutes (而不是 Restarting)
sudo docker logs --tail 30 navi
# 期望: listening on [::]:80 / auth: ENABLED (ttl 72h)
# 若是 Restarting: 看日志中的报错 (最常见是 Cannot find module './discovery')
```

2 健康检查接口

```
curl -s http://127.0.0.1:8080/api/health
# 期望输出: {"ok":true,"ipv6":true,"authEnabled":true}
```

`authEnabled:true` 说明密码保护已生效。若为 `false`，说明 `NAVI_PASSWORD` 没被读到——按顺序用 `docker compose config` 检查变量是否解析、`docker inspect navi` 确认环境变量已注入。

3 从另一台设备访问

```
curl -s -o /dev/null -w "%{http_code}\n" http://NAS的IP:8080/
# 期望: 302 (未登录跳转登录页)
curl -s -o /dev/null -w "%{http_code}\n" http://NAS的IP:8080/api/config
# 期望: 401 — 这是正确行为，不是故障！
```

别把 401 当成部署失败。启用密码保护后，所有 API 与页面在未登录时都会拒绝访问（页面 302 跳登录页、API 返回 401）。这正是安全设计生效的表现。要判断是否正常，只看上面那条 `/api/health`。

4 浏览器完整验证

1. 打开 `http://NAS的IP:8080` → 应自动跳到登录页
2. 输入密码（若设了 `NAVI_USERNAME` 则还需输入用户名）→ 进入导航首页
3. 点右上角「编辑」→ 底部出现保存栏 → 随意拖动一张卡片或改个名称 → 点「保存」
4. 重点验证「保存」是否真的成功（这是整目录挂载是否配对的判定点）。保存成功后刷新页面，改动仍在，说明 `data/config.json` 已正确写入宿主机
5. 进编辑模式 → 试一次「上传」Logo 或「图床库」批量上传 → 刷新后图片仍能显示，说明 `data/uploads/` 持久化正常

5 宿主机侧确认数据落盘

```
ls -l /vol1/1000/docker/navi/data/  
# 应看到 config.json 与 uploads/ 目录；刚保存过的话 config.json 的修改时间是刚刚
```

12. 常见问题与排查

现象	原因	处理
容器 Restarting，日志报 <code>Cannot find module './discovery'</code>	源码漏传 <code>discovery.js</code> （它在项目根目录，不在 <code>public/</code> 里）	补传该文件后 <code>docker compose up -d --build</code> 重建
<code>compose up</code> 直接报 <code>NAVI_PASSWORD is required</code>	缺少 <code>.env</code> 或其中没写该变量（这是刻意设计的强校验）	创建 <code>.env</code> 并写入 <code>NAVI_PASSWORD=...</code> ；或在 GUI 里把密码直接写进环境变量
网页点「保存」永远失败 / 报错	用了单文件挂载 <code>config.json</code> ，跨挂载点 <code>rename</code> 在飞牛卷上触发 <code>EBUSY</code>	改为整目录挂载 <code>/vol1/1000/docker/navi/data:/app/data</code> 。已有旧数据可用仓库脚本一键迁移： <code>sudo bash scripts/migrate-to-dir-mount.sh</code>
日志有 <code>EBUSY / EXDEV</code> 但仍提示保存成功	服务内置了写入降级兜底：原子写失败时自动改为直接覆盖写入	属正常降级（牺牲一点原子性保可用）。仍建议改整目录挂载以恢复原子性
构建卡在 <code>Pulling node:22-alpine</code>	NAS 访问 Docker Hub 网络不通	飞牛 Docker 设置里配镜像加速；或改用「跨机构建 + <code>docker save/load</code> 」

页面打不开，但容器是 Up	端口映射对不上，或宿主端口被占用	<code>sudo ss -tlnp grep 8080</code> 查占用；确认 compose 里 <code>ports</code> 左侧数字与访问端口一致； 不要改容器内 PORT
能打开登录页，但密码正确也进不去	用了 <code>NAVI_PASSWORD_HASH</code> 但哈希算错（漏了 <code>navi-site:</code> 前缀）	按第 5 节第 4 步的带前缀命令重新生成。裸 sha256 会稳定 401 且无任何报错
浏览器点登录没反应 / 登录后立刻跳回登录页	Cookie 被拒（HTTP/HTTPS 混用、跨域反代丢 Host）	统一用同一协议访问；反代时保留 Host 头并回传 <code>Set-Cookie</code>
重启容器后需要重新登录	正常行为： 会话签名密钥每次启动随机生成，旧会话全部失效	无需处理。这是安全设计，重新登录即可
飞牛「文件管理」无法编辑 <code>data/</code> 里的文件	容器以 root 运行，写入的文件属主为 <code>root:root</code>	<code>sudo chown -R 1000:1000 /vol1/1000/docker/navi/data</code> ；或给 compose 加 <code>user: "1000:1000"</code> （见第 7 节）
上传的 Logo 刷新后 404 / 图片丢失	<code>NAVI_UPLOAD_DIR</code> 指到了未挂载的容器内路径	确认它是 <code>/app/data/uploads</code> ，且 <code>/app/data</code> 已挂载到宿主机目录
「服务发现」弹窗提示无法访问 Docker	未挂载 socket，属预期降级	按第 10 节挂载 socket 并重建容器；或在无 Docker 环境下接受此降级（其余功能不受影响）
弹窗报 扫描失败：服务发现失败：Protocol "https:" not supported. Expected "http:"	旧版 <code>discovery.js</code> 用 <code>http.request</code> 去请求 https 的图标 CDN，Node 会 同步抛错 （错误回调接不住），把整次服务发现打挂	更新到已修复版本（现按协议自动选 <code>http/https</code> ，且单点失败只降级不中断）；临时规避可设 <code>NAVI_ICON_PROBE=0</code>
服务发现弹窗转圈很久才出结果	NAS 访问不了图标 CDN（jsDelivr）时，每个服务都要等满探测超时	设 <code>NAVI_ICON_PROBE=0</code> 直接关闭在线探测（卡片回退字母图标，功能不受影响）；或把 <code>NAVI_ICON_PROBE_BASE</code> 指向自建镜像
服务发现列出的地址 IP 不对	自动探测到的网卡不是你想要的那块（NAS 常有多网口 / 虚拟网卡）	显式设置 <code>NAVI_LAN_HOST=NAS的局域网IP</code>
导入自己刚导出的备份 ，弹窗提示「文件完整性校验失败（数据可能被篡改或损坏）」	旧版前端把「本地算不出哈希」误判成「校验不通过」。浏览器只在 安全上下文 （HTTPS 或	更新到已修复版本并 重建镜像 （修复后本地算不出哈希会跳过预检，直接交由后端权威校验，篡改文件仍会被拦下）

	localhost) 才提供 crypto.subtle, 而经 http://局域网IP:18880 访问时它不存在 → 文件明明没坏却导不回来。后端校验一直是正常的	
代码明明改了、镜像也重建了, 页面还是老样子	浏览器对 js/css 有强缓存, 缓存键是 URL (含 ?v=)。旧版靠手工改 index.html 里的 ?v=, 一旦漏改, 浏览器就继续用旧文件, 与服务器无关	先跑 node scripts/check-deploy.cjs http://NAS的IP:端口 你的密码 判定容器里到底是不是新代码 (退出码 0=新 / 1=旧)。若第 3 列判定为新: 强制刷新 (Ctrl+F5) 即可。若判定为旧: 多半是只替换了部分文件 (server.js、discovery.js、public/ 必须一起覆盖) 或没重建镜像/容器
GUI 编辑器给某几行画红线, 确认时提示 bad indentation of a mapping entry / did not find expected key	YAML 缩进不齐: 取消注释时多删或少删了空格, 或这一行比同级的其它行缩进更深/更浅	同级 - 必须严格对齐同一列 (本项目原版是 6 空格 + - + 空格)。按第 6 节「YAML 缩进铁律」逐行核对, 最稳的做法是直接把那两行重新注释掉 (LAN 地址会自动探测)
容器内变量值变成了「好几行拼在一起」的怪字符串 (如 NAVI_UPLOAD_DIR=/app/data/uploads - NAVI_LAN_HOST=...)	缩进比同级更深时, YAML 会把它当成上一条的多行纯量折进去——这种写法不报错但变量静默失效, 比语法报错更危险	把该行缩进改回与同级一致。改完可在 SSH 里用 docker compose config 先本地校验一眼环境变量展开结果
图标显示为字母方块	图标来自在线 CDN, NAS 或客户端无法访问外网时加载失败, 自动回退	属正常回退。可改从本地图床库选用图标, 完全离线可用

13. 日常运维

操作	命令 / 做法
查看实时日志	sudo docker logs -f navi
停止 / 启动 / 重启	sudo docker compose stop · start · restart
备份数据 (推荐)	sudo tar czf /vol1/1000/navi-backup-\$(date +%F).tar.gz -C /vol1/1000/docker/navi data 网页上的「备份」按钮只导出 config.json, 不含图片, 所以这里用整目录打包
恢复数据	停容器 → 用备份覆盖 data/ → 起容器

改密码 / 改端口	改 <code>.env</code> 或 <code>compose</code> 后: <code>sudo docker compose up -d --force-recreate</code> 飞牛 GUI 则按「停止 → 改 <code>yml</code> → 清除项目 → 再启动」
升级到新版源码	<code>git pull</code> (或覆盖新源码) → <code>sudo docker build -t navi:latest .</code> → <code>sudo docker compose up -d --force-recreate</code> 。配置在宿主机挂载, 重建不丢数据 务必把 <code>server.js</code>、<code>discovery.js</code>、<code>public/</code> 一起覆盖 ——漏传任何一个都会表现为「改了没效果」
确认新代码真的生效	在电脑上跑部署判定探针: <code>node scripts/check-deploy.cjs http://NAS的 IP:端口 你的密码</code> 退出码 <code>0</code> =已是新代码, <code>1</code> =容器里还是旧代码 (会打印该改什么), <code>2</code> =检查没跑完 (端口不通 / 密码不对) 密码不对也能用 : 探针会退化为「免登录判定」, 凭公开的 <code>/login.html</code> 响应头先给 <code>server.js</code> 定性, 并明确标注 <code>app.js</code> 那项未验证——不会假装通过 也可直接查容器内文件, 用 <code>wc -c</code> 而不是 <code>ls -l</code> : <code>sudo docker exec navi wc -c /app/server.js /app/discovery.js /app/public/js/app.js</code> 输出三行短数字 (<code>ls -l</code> 在窄终端里会被折行搞乱, 看不出哪个对哪个), 与本地源码字节数逐一比对即可
彻底卸载	<code>sudo docker compose down</code> → <code>sudo docker rmi navi:latest</code> 。数据目录 <code>data/</code> 会保留, 确认无用后再手动删除

回滚思路: 升级前先执行上面的 `tar` 备份, 并把旧的 `navi:latest` 打个标记 (`docker tag navi:latest navi:backup`)。出问题 `docker tag navi:backup navi:latest` 再重建容器即可回到旧版本——因为数据在宿主机挂载里, 镜像回滚不影响数据。

14. 部署检查清单

- ☐ Docker 应用已安装, 并已选定 SSD 存储空间
- ☐ 目录 `/vol1/1000/docker/navi/` 已建, 源码五项齐全 (含 `discovery.js`)
- ☐ `data/config.json` 已初始化, 且是合法 JSON
- ☐ `.env` 已写入强密码 (或已填 `NAVI_PASSWORD_HASH`)
- ☐ 挂载是**整目录** `data:/app/data`, **不是**单文件挂载
- ☐ 宿主机 8080 端口未被占用 (或已改用其它端口)
- ☐ 镜像已构建/导入, `docker images` 能看到 `navi:latest`
- ☐ `docker compose ps` 显示 Up 且未重启
- ☐ `/api/health` 返回 `authEnabled:true`
- ☐ 浏览器能登录, 且「保存」成功、刷新后改动仍在
- ☐ 已做一次整目录 `tar` 备份
- ☐ (可选) 已挂 Docker Socket 并确认接受其安全影响

□ (可选) 已配反向代理 + HTTPS, 未把 8080 直接暴露公网

15. 命令速查 (可直接整段粘贴)

假设源码已放到 `/vol1/1000/docker/navi/`, 以下是完整的一次性部署命令:

```
# ---- 1. 建目录 ----
sudo mkdir -p /vol1/1000/docker/navi/data/uploads
cd /vol1/1000/docker/navi

# ---- 2. 初始化配置 ----
cp public/config.example.json data/config.json

# ---- 3. 写密码 (把 CHANGE_ME 换成你的强密码) ----
cp .env.example .env
sed -i 's/^NAVI_PASSWORD=.*\/NAVI_PASSWORD=CHANGE_ME/' .env

# ---- 4. 构建镜像 ----
sudo docker build -t navi:latest .

# ---- 5. 启动 ----
sudo docker compose up -d

# ---- 6. 验证 ----
sudo docker compose ps
curl -s http://127.0.0.1:8080/api/health
sudo docker logs --tail 20 navi

# ---- 7. 权限 (让文件管理也能操作数据目录) ----
sudo chown -R 1000:1000 /vol1/1000/docker/navi/data
```

第 3 步的 `sed` 会把密码明文写进 shell 历史。介意的话改用 `vi .env` 手动编辑, 或在命令前加一个空格 (若 shell 配了 `HISTCONTROL=ignorespace`)。